Cairo University - Faculty Of Engineering
Computer Engineering Department
ML - 2026

Phishing Website Detection Based on ML

Project Document

Submitted to

Dr. Yahia Zakaria

Eng. Abdelrahman Kaseb

Submitted by Team:

Name	ID
Mohamed Sayed	9220695
Abdallah Salah	9220478
Mohamed Abdelrahman	9211015
Abdelrahman Samy	9220430

Contents

1 Problem Definition

The problem is a binary classification of URLs to determine if a website is a legitimate or phishing attempt. Phishing is an attack where an attacker mimics trusted websites to steal user data (credentials, financial info). Specifically, this project aims to take a high-dimensional dataset of URL-based features and classify them without using deep learning, focusing on the efficiency of classical algorithms.

2 Motivation

We searched for many ideas with the goal of finding one that is not trivial. Phishing Website Detection is a real-world application with ongoing research, making it a challenging and non-trivial task for the following reasons:

1. **Imbalanced Datasets:** The number of legitimate websites is more than phishing attempts. Handling this skewness requires techniques to ensure the model doesn't simply guess the majority class.
2. **Overfitting Problems:** Because phishing datasets often contain a high number of features (111 in our selected dataset), this requires good application of Generalization, Validation, and Regularization to ensure the model performs well on unseen data.
3. **Non-Linearity:** Phishing URLs are designed to mimic legitimate ones using a lot of tricks. Detecting these patterns requires non-linear decision boundaries, which provides an excellent opportunity to apply Kernelized SVMs and Ensemble Learning (Boosting/Bagging).
4. Attackers constantly change their tactics to bypass detection. This makes the Bias-Variance Tradeoff important to our project. We need a model complex enough to detect new threats (low bias) but stable enough not to react to harmless URL variations (low variance).

3 Evaluation Metrics

To ensure a good analysis of the model performance, we will use:

Accuracy: To measure the overall percentage of correct predictions.

Precision: To minimize "False Positives" (blocking a legitimate site, which frustrates users).

Recall: To minimize "False Negatives" (missing a phishing site, which leads to security breaches).

F1-Score: To provide a balanced metric between Precision and Recall.

ROC-AUC Curve: To evaluate the model's ability to distinguish between classes across various probability thresholds.

4 Dataset

- Vrbancic, G., Fister Jr, I., & Podgorelec, V. (2020). Datasets for phishing websites detection. *Data in Brief*.
- Dataset accessability link: https://data.mendeley.com/datasets/72ptz43s9v/1

References

- Bahaghighat, M., Ghasemi, M., & Ozen, F. (2023). A high-accuracy phishing website detection method based on machine learning. *Journal of Information Security and Applications*.
- Omari, K., & Oukhatar, A. (2025). Advanced Phishing Website Detection with SMOTETomek-XGB: Addressing Class Imbalance for Optimal Results. *Procedia Computer Science*, 252, 289–295.
- Birthriya, S. K., Ahlawat, P., & Jain, A. K. (2025). Phishing Website Detection with XGBoost and Adaptive Hyperparameter Optimization using the Bat Algorithm. *Procedia Computer Science*, 258, 1774–1782.

5 Dataset Analysis and Preprocessing

The shape of the raw dataset is (88647, 112), there is no null values, and there are 1438 duplicate rows. Based of the below class distribution which shows a relative imbalance in our dataset we decided to use SMOTE for synthetic oversampling to balance our dataset.

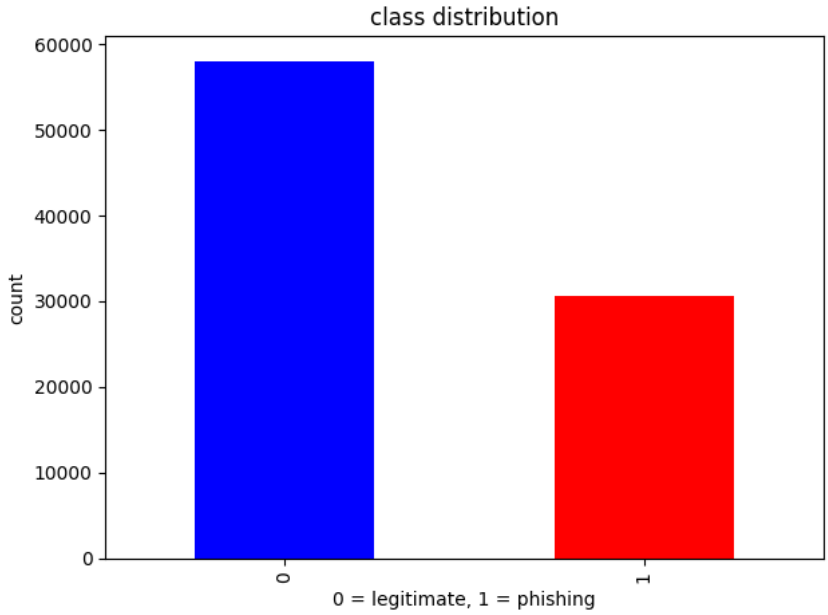

Figure 1: Raw dataset class distribution

There are 111 features and those features are categorized as follows:

- attributes based on the whole URL properties
- attributes based on the domain properties
- attributes based on the URL directory properties
- attributes based on the URL file properties
- attributes based on the URL parameter properties
- attributes based on the URL resolving data and external metrics

---

There are some constant features that we should remove to eliminate redundancy and those features won't help models in learning phishing and non-phishing URLs.

- Number of constant features: 13
- Number of quasi-constant features (99% of data have same value): 16

There are 0 categorical features so no need for one-hot encoding, and all features are numeric either binary or continuous.

Preprocessing steps before spliting the dataset is as follows:

1. Remove Duplicate examples except for the highest correlated example with target
2. Remove constant features
3. Remove quasi-constant features (99%)

6 Outliers and Multicolinearity

To be honest we are shocked of how many outliers in our dataset using IQR. The large amount of outliers in security, fraud, phishing datasets is expected and with 88k samples having thousands of outliers means these features strongly differentiate phishing from legitimate sites, in cybersecurity ML these outliers are valuable signals not noise.

Top 5 Features with the most Outliers:

- qty_dot_url: 34486 outliers
- domain_spf: 27763 outliers
- file_length: 21288 outliers
- qty_ip_resolved: 15452 outliers
- qty_hyphen_url: 15384 outliers

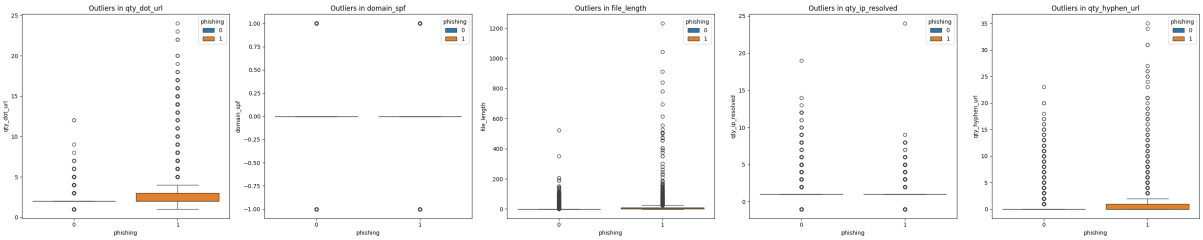

Figure 2: Top 5 Features with the most Outliers

Now lets talk about Multicolinearity, our dataset has a correlation explosion and the reason for this is:

Example URL

```
https://www.example.com/path/file.html?param=value
```

Feature definitions:

```
qty_dot_url        = number of dots in the entire URL       (e.g., 4)
qty_dot_domain     = number of dots in the domain only      (e.g., 2)
qty_dot_directory  = number of dots in the path only        (e.g., 1)
qty_dot_file       = number of dots in the filename only    (e.g., 1)
```

so we can conclude that

```
qty_dot_url = qty_dot_domain + qty_dot_directory + qty_dot_file
```

therefore the _url version of a feature will always be highly correlated with its _domain, _directory, and _file
because:

- each qty_* feature is repeated across 4 different URL parts
- there are 15+ character types

This result in large number of correlated features leading to a **correlation explosion** but now we have strong view about this correlation. In experiments section we will try to know is removing correlated features better than keeping them?

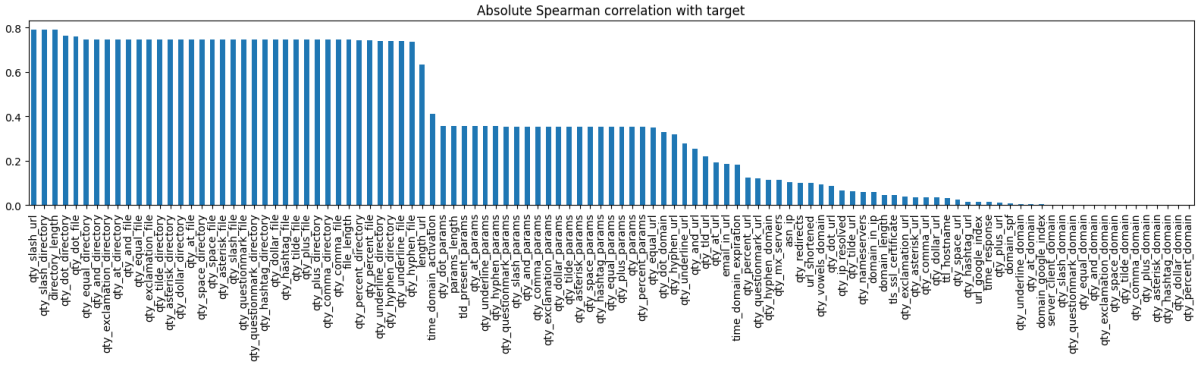

Figure 3: Feature-target absolute correlation

The above graph illustrates the most correlated features with the target also if we observe the right most features we can see that the correlation with target is approximately zero and those features are the constant features.

7 ZeroR Classifier (baseline performance)

- Majority class: 0 (0=Legitimate, 1=Phishing) — ZeroR Accuracy: 0.6503

8 Selected Models

- Logistic Regression — SVM — Random Forest (Bagging) — XGBoost (Boosting)

9 Experiments Setup

Through our experiments we tried to answer the following questions:

1. Is dropping correlated features to eliminate multicollinearity would be better than keeping them?
2. Does PCA will help with linear models like LR and SVM?
3. Does balancing the dataset using SMOTE will help to achieve higher score?

DCF = Drop Correlated Features

Scenario for LR and SVM	**PCA**	**DCF**	**SMOTE**
S1	0	0	0
S2	0	0	1
S3	0	1	0
S4	0	1	1
S5	1	0	0
S6	1	0	1

Note that we doesn't do PCA and DCF in the same experiment because they both try to solve the same problem which is eliminate correlated features and have strong independent features.

Scenarios for RF and XGBoost	SMOTE
S1	0
S2	1

Note that for ensemble learning we do not consider DCF or PCA because ensemble learning can handle correlated features internally.

10 Preventing Data Leakage

1. Removing Duplicates from Dataset
Removing duplicates is the first obvious action we take to prevent data leakage, because the model may be trained on examples that also exist in the test set which can cause fake scoring and ranking for our models.

2. Train/CV/Test split happens before everything else
Nothing from scaling, correlation filtering, SMOTE touch the data before this split. CV and test sets are immediately separated and never used to make any fitting decision.

3. StandardScaler is fit on train only
If we call fit_transform on CV or test their mean and variance would contribute into scaling parameters meaning the model indirectly sees information about those splits during training. Using transform only prevents this.

4. Correlation filter is computed on train only
If we compute correlation on the full dataset before splitting, feature selection would be influenced by patterns in CV and test sets. The correlation of CV and test is completely invisible to the filter.

5. SMOTE and PCA are inside the pipeline
SMOTE is inside the pipeline and applied within each CV fold. The core problem is SMOTE creates synthetic samples using real data. So the danger is what if synthetic samples contain information from validation data?

WRONG WAY:

X_train → SMOTE → X_resampled → split into folds
Now inside a fold:
the train fold contains synthetic samples that were created using validation fold data!
Why is this BAD? Because the model indirectly sees validation data during training

---

CORRECT WAY (We are using imblearn.pipeline)

Pipeline:
——SMOTE
——model
GridSearchCV(pipeline)

What happens internally? Now GridSearchCV does:
For each fold:
——sub-train → apply SMOTE HERE
——sub-val → untouched!

Similarly, PCA is inside the pipeline for the same reason. PCA computes principal components from the data it sees. If PCA were fit on the full dataset before splitting:
Full dataset → PCA.fit_transform() → split into train/cv/test
Now the principal components contain variance information from CV and test sets — the model indirectly learns the structure of data it should never have seen.

6. Leakage-Free Pipeline
The most convincing evidence that our pipeline is leakage free is the consistency between CV and test scores across all 16 experiments we will see this consistency in results section.

11 Hyperparameter Tuning

We need to find the optimal configurations of each model before it sees the test data. Unlike model parameters which learned during training, hyperparameters are choosen and set before training starts and they control the learning process for example: control the depth of decision trees or control the regularization.

We use **GridSearchCV** from scikit-learn which is an exhaustive way of trying every combination of hyperparameters and choose the best combination that gives the highest cross-validation score. So we need to define a grid for each model so the GridSearchCV could test all combinations.

We choose **GridSearchCV** for the following reasons:

- **Exhaustive**: Every combination in the grid is evaluated
- **Cross-validation**: Each combination is evaluated using K-fold cross-validation not on a single train/val split producing more reliable estimates
- **Pipeline**: GridSearchCV works very good with `imblearn.pipeline.Pipeline` meaning SMOTE and PCA are applied correctly inside each fold rather than leaking across folds
- **Refit**: After finding the best hyperparameters, GridSearchCV automatically refits the best configuration on the entire training set (`refit=True`), producing the final model used for evaluation

We use **StratifiedKFold** with $k = 5$ folds inside GridSearchCV. Stratified KFold ensures that each fold contains the same class distribution as the original training set this is important for imbalanced classification.

The scoring metric used to select the best hyperparameters is **F1 score**. This means GridSearchCV selects the configuration that maximize F1 not accuracy to prevent it from selecting configurations that perform well on the majority class only

Each model's grid was designed using two rules:

1. **Target the bias-variance trade-off**: Parameters that control model complexity are prioritized. After initial experiments revealed overfitting (train error ≈ 0, CV gap ≈ 0.04), grids were redesigned to include stronger regularization values
2. **Respect resources cabability**: Each grid is constrained to produce a reasonable number of fits. The total fit count for each model is:

$$\text{Total fits} = \text{combinations} \times k$$

Logistic Regression Grid

LR uses the `saga` solver to support both L1 and L2 regularization. The grid is split into two sub grids:

```
LR_GRID = [
    {"model__C": [0.01, 0.1, 1, 10, 100], "model__l1_ratio": [0.0], "model__max_iter": [1000]},
    {"model__C": [0.01, 0.1, 1, 10, 100], "model__l1_ratio": [1.0], "model__max_iter": [1000]},
]
```

`C` controls regularization strength (smaller = stronger)
Total fits: 50

LinearSVC Grid

LinearSVC is wrapped with `CalibratedClassifierCV` to enable `predict_proba` so parameters use the `model__estimator__` prefix:

```
LINEAR_SVC_GRID = [
    {
     "model__estimator__C": [0.01, 0.1, 1, 10, 100],
     "model__estimator__penalty": ["l2"],
     "model__estimator__loss": ["squared_hinge"],
     "model__estimator__max_iter": [2000]
    },
    {
     "model__estimator__C": [0.01, 0.1, 1, 10, 100],
     "model__estimator__penalty": ["l1"],
     "model__estimator__loss": ["squared_hinge"],
     "model__estimator__dual": [False],
     "model__estimator__max_iter": [2000]
    }
]
```

Total fits: 50

Random Forest Grid

The initial grid caused overfitting (train error ≈ 0) so it was redesigned with stronger regularization:

```
RF_GRID = {
    "model__max_depth": [8, 12, 16],
    "model__min_samples_split": [10, 20],
    "model__min_samples_leaf": [4, 8],
    "model__n_estimators": [300],
    "model__max_features": ["sqrt", "log2"],
}
```

Key decisions:

- Removed `max_depth=None` to avoid memorization
- Increased `min_samples_leaf` to prevent single sample leaves
- Fixed `n_estimators=300` to reduce search space

Total fits: 120

XGBoost Grid

The initial XGBoost model showed overfitting (train error ≈ 0 and CV gap of 0.040) so the grid was updated to control model complexity and improve generalization:

```
XGB_GRID = {
    "model__max_depth":        [4, 6],
    "model__min_child_weight": [5, 10],
    "model__gamma":            [0.1, 0.5],
    "model__reg_lambda":       [5.0, 15.0],
    "model__learning_rate":    [0.05, 0.1],
    "model__n_estimators":     [500],
    "model__subsample":        [0.8],
    "model__device":           ["cuda"],
}
```

Key ideas (simple explanation):

- **max_depth [4, 6]**: smaller trees → less overfitting
- **min_child_weight [5, 10]**: avoid splits on very small data → reduces noise
- **gamma [0.1, 0.5]**: only allow useful splits → acts like pruning

- **reg_lambda [5.0, 15.0]**: stronger regularization → reduces overconfident predictions
- **learning_rate [0.05, 0.1]**: slower learning → better generalization
- **n_estimators = 500**: enough trees for stable performance
- **subsample = 0.8**: use part of data per tree → reduces overfitting

Overall, the goal of this grid is to make the model simpler and prevent memorization
Total fits: 160

Experiment Execution Steps

The full experiment pipeline runs as follows:

1. **Load and split data**

   ```
   X_train, X_cv, X_test, y_train, y_cv, y_test = load_splits(DATA_PATH)
   ```

   Stratified 60/20/20 split. All subsequent steps use only `X_train` for fitting decisions.

2. **Prepare feature arrays**

   ```
   arrays = experiments_data(X_train, X_cv, X_test, y_train)
   ```

   Computes correlation filter (on train only), fits both StandardScalers (on train only), and returns three array variants: full scaled, DCF scaled, and raw (for trees).

3. **Build pipeline per scenario**

   ```
   pipeline = build_pipeline(model, use_pca, use_smote)
   ```

   Returns an `imblearn.Pipeline` with steps ordered as: SMOTE → PCA → model. Only the requested steps are included.

4. **Run GridSearchCV**

   ```
   grid = GridSearchCV(
       estimator  = pipeline,
       param_grid = param_grid,
       cv         = CV,
       scoring    = "f1",
       n_jobs     = -1,
       refit      = True,
   )
   grid.fit(Xtr, y_train)
   ```

   For each hyperparameter combination: fits the pipeline on 4 sub-folds, evaluates on the 5th, averages F1 across 5 folds. Selects the best combination and refits on the full `X_train`.

5. **Evaluate on all splits**

   ```
   results = evaluate_fitted(
       grid.best_estimator_,
       Xtr, y_train,
       Xcv, y_cv,
       Xte, y_test
   )
   ```

Reports F1, ROC-AUC, Precision, Recall, and Accuracy on train, CV, and test sets separately.

6. **Save model and results**

```
joblib.dump(grid.best_estimator_, f"models/{key}.joblib")
summary_df.to_csv("results/summary.csv")
```

Each of the 16 best estimators is saved independently for later use in learning curve analysis, runtime comparison, and API deployment.

Steps 3–6 repeat for all 16 experiment combinations (6 scenarios × 2 linear models + 2 scenarios × 2 tree models), producing comparable results table across all models and preprocessing configurations

12 Results and Comparisons

Table **??** presents all 16 experiments sorted by test F1 score.

Experiment	CV F1	T-F1	T-F1 Tuned	Threshold	AUC	Precision	Recall	Accuracy
XGB__S2	0.9572	0.9572	0.9576	0.40	0.9951	0.9497	0.9657	0.9701
XGB__S1	0.9569	0.9579	0.9590	0.43	0.9952	0.9530	0.9651	0.9712
RF__S1	0.9452	0.9458	0.9452	0.54	0.9929	0.9491	0.9413	0.9618
RF__S2	0.9452	0.9431	0.9451	0.63	0.9926	0.9512	0.9392	0.9619
LINEAR_SVC__S1	0.9027	0.8981	0.8980	0.51	0.9767	0.8925	0.9036	0.9282
LR__S1	0.9025	0.8979	0.8978	0.51	0.9768	0.8909	0.9049	0.9280
LR__S2	0.9024	0.8915	0.8967	0.65	0.9767	0.8897	0.9038	0.9272
LINEAR_SVC__S2	0.9021	0.8887	0.8969	0.67	0.9764	0.8897	0.9042	0.9273
LR__S4	0.8995	0.8969	0.8969	0.50	0.9733	0.8826	0.9118	0.9267
LR__S3	0.8979	0.8814	0.8976	0.35	0.9730	0.8815	0.9144	0.9271
LINEAR_SVC__S4	0.8972	0.8962	0.8953	0.51	0.9732	0.8832	0.9077	0.9258
LINEAR_SVC__S3	0.8970	0.8807	0.8963	0.33	0.9731	0.8743	0.9193	0.9256
LR__S5	0.8969	0.8914	0.8915	0.51	0.9740	0.8795	0.9038	0.9231
LINEAR_SVC__S5	0.8962	0.8915	0.8899	0.53	0.9739	0.8861	0.8938	0.9227
LINEAR_SVC__S6	0.8915	0.8805	0.8877	0.69	0.9726	0.8824	0.8931	0.9210
LR__S6	0.8912	0.8834	0.8881	0.66	0.9726	0.8771	0.8995	0.9208

Table 1: Model Performance Comparison

Experiment	Best Params
XGB__S2	{'model__device': 'cuda', 'model__gamma': 0.5, 'model__learning_rate': 0.1, 'model__max_depth': 6, 'model__min_child_weight': 5, 'model__reg_lambda': 5.0, 'model__subsample': 0.8}
XGB__S1	{'model__device': 'cuda', 'model__gamma': 0.1, 'model__learning_rate': 0.1, 'model__max_depth': 6, 'model__min_child_weight': 5, 'model__reg_lambda': 5.0, 'model__subsample': 0.8}
RF__S1	{'model__max_depth': 16, 'model__max_features': 'sqrt', 'model__min_samples_leaf': 4, 'model__min_samples_split': 10, 'model__n_estimators': 500}
RF__S2	{'model__max_depth': 16, 'model__max_features': 'sqrt', 'model__min_samples_leaf': 4, 'model__min_samples_split': 10, 'model__n_estimators': 500}
LINEAR_SVC__S1	{'model__estimator__C': 100, 'model__estimator__loss': 'squared_hinge', 'model__estimator__max_iter': 2000, 'model__estimator__penalty': 'l2'}
LR__S1	{'model__C': 1, 'model__max_iter': 2000, 'model__penalty': 'l1', 'model__solver': 'saga'}
LR__S2	{'model__C': 10, 'model__max_iter': 2000, 'model__penalty': 'l1', 'model__solver': 'saga'}
LINEAR_SVC__S2	{'model__estimator__C': 100, 'model__estimator__loss': 'squared_hinge', 'model__estimator__max_iter': 2000, 'model__estimator__penalty': 'l2'}
LR__S4	{'model__C': 1, 'model__max_iter': 2000, 'model__penalty': 'l1', 'model__solver': 'saga'}
LR__S3	{'model__C': 10, 'model__max_iter': 2000, 'model__penalty': 'l1', 'model__solver': 'saga'}
LINEAR_SVC__S4	{'model__estimator__C': 0.01, 'model__estimator__loss': 'squared_hinge', 'model__estimator__max_iter': 2000, 'model__estimator__penalty': 'l2'}
LINEAR_SVC__S3	{'model__estimator__C': 0.1, 'model__estimator__loss': 'squared_hinge', 'model__estimator__max_iter': 2000, 'model__estimator__penalty': 'l2'}
LR__S5	{'model__C': 1, 'model__max_iter': 2000, 'model__penalty': 'l1', 'model__solver': 'saga'}
LINEAR_SVC__S5	{'model__estimator__C': 10, 'model__estimator__loss': 'squared_hinge', 'model__estimator__max_iter': 2000, 'model__estimator__penalty': 'l2'}
LINEAR_SVC__S6	{'model__estimator__C': 0.1, 'model__estimator__loss': 'squared_hinge', 'model__estimator__max_iter': 2000, 'model__estimator__penalty': 'l2'}
LR__S6	{'model__C': 1, 'model__max_iter': 2000, 'model__penalty': 'l1', 'model__solver': 'saga'}

Table 2: Best Hyperparameters per Experiment

13 Overfitting and Evidence of Generalization

In this project we address overfitting through four mechanisms: regularization via hyperparameter tuning, constraints in tree models, cross-validation during model selection, and a leakage free pipeline design.

The most direct evidence of generalization is the consistency between CV F1 and Test F1 across all 16 experiments.

A leaky or overfit pipeline almost always produces CV F1 $\gg$ Test F1 because the model has indirectly seen validation data during training. The near zero gaps here confirm that all four leakage prevention mechanisms are working correctly.

14 Bias-Variance Analysis

We use learning curves — plots of training error and CV error (both computed as $1-\text{F1}$) against training set size — to diagnose the bias-variance

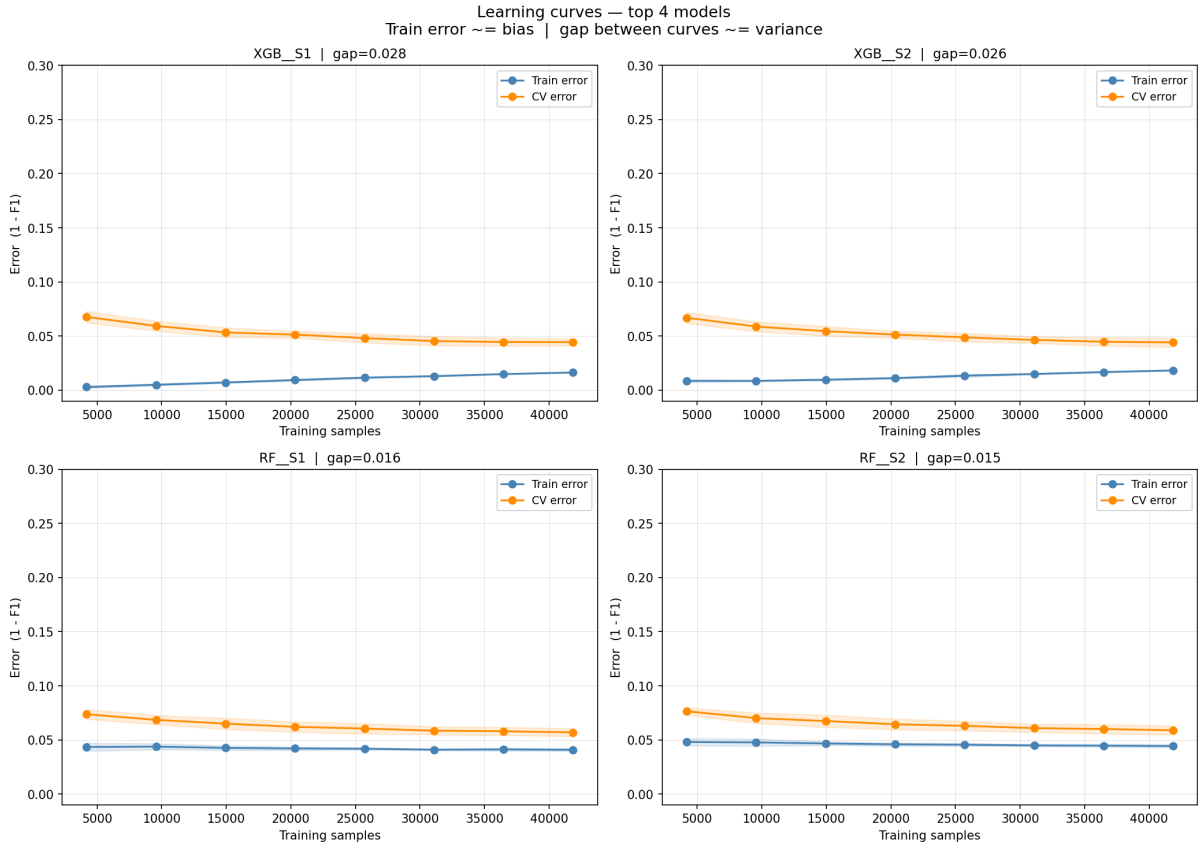

Figure 4: Learning Curves for Top 4 Experiments

Four patterns can appear in learning curves:

Pattern	Train error	CV error	Diagnosis
Both high, converging	High	High	High bias (underfitting)
Train low, large gap	Near 0	High	High variance (overfitting)
Both low, converging	Low	Low	Well generalised
Both flat, small gap	Low	Low	Optimal — more data won't help

Table 3: Learning curve pattern

15 Conclusion

In this project, we tested four models (Logistic Regression, LinearSVC, Random Forest, and XGBoost) for phishing URL detection using different preprocessing scenarios.

The main results are:

- **XGBoost is the best model**: It achieved the highest performance (F1 = 0.9576) with very small difference between CV and test so it generalizes well
- **The following setup works best**: Using SMOTE, no PCA, or correlation filtering gave the best results. PCA reduces useful information and models can handle correlated features by themselves
- **Linear models are limited**: Logistic Regression and LinearSVC could not go above F1 $\approx$ 0.89
- **Regularization improved models**: It reduced overfitting in tree models with only a very slight drop in performance

16 Model Deployment: Microsoft Edge Extension

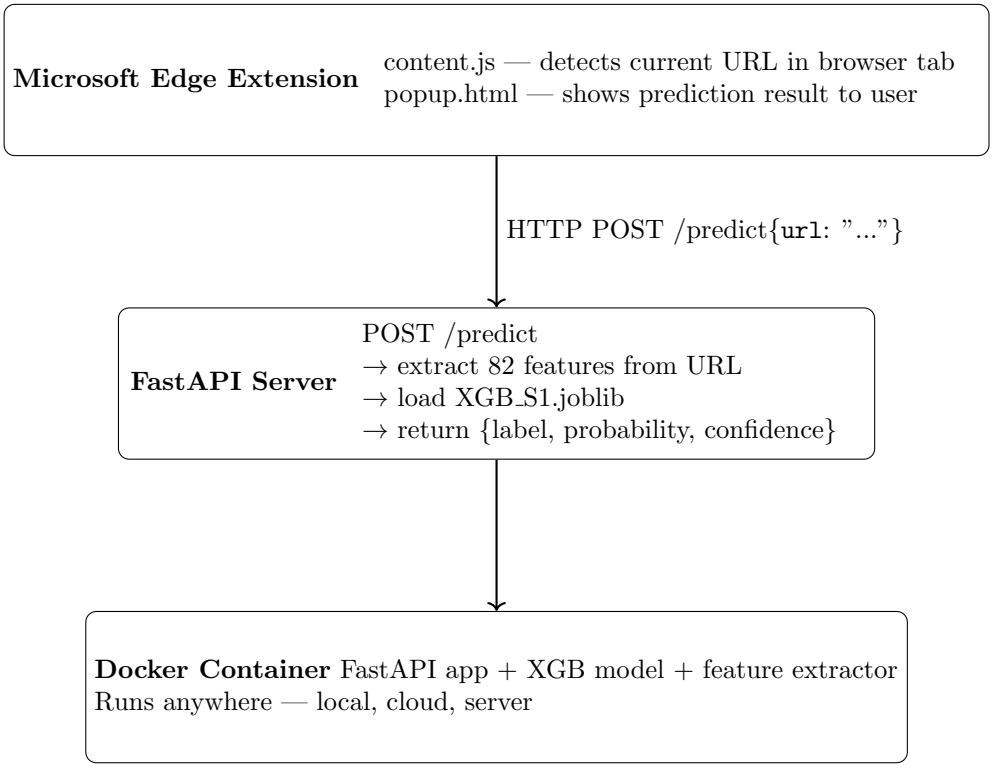

Figure 5: System Architecture of Phishing Detection Extension